

2025년 상반기 K-디지털 트레이닝

멀티 스레드

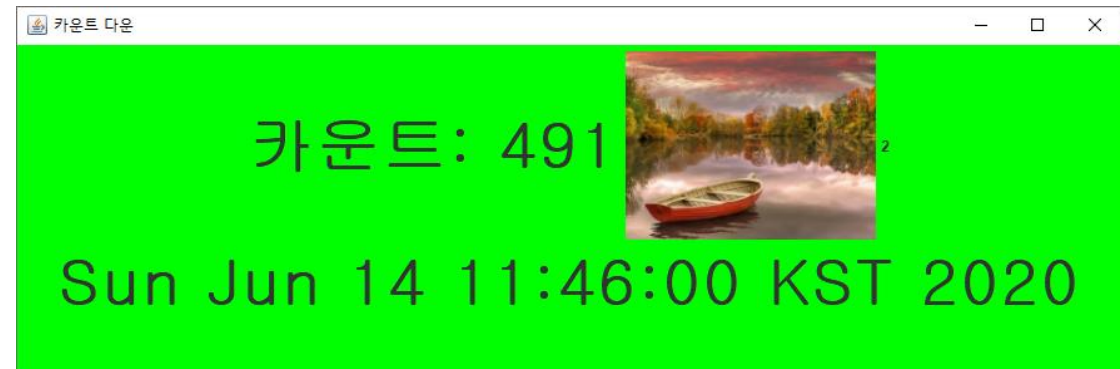
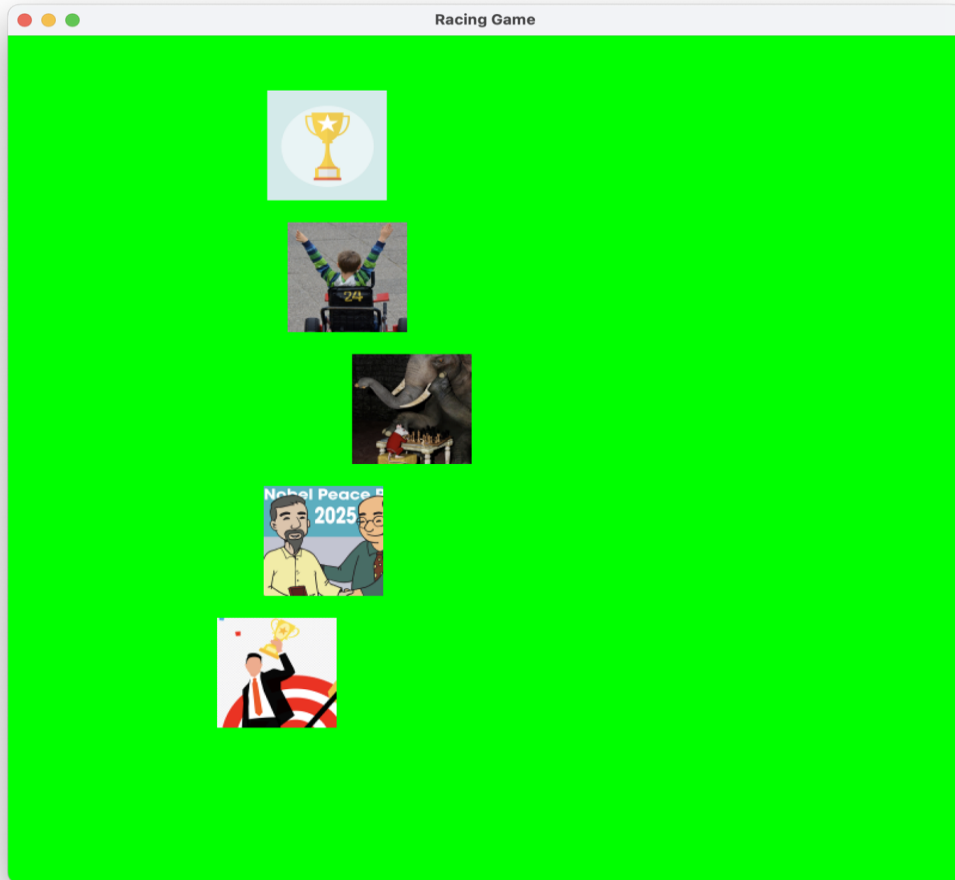
[KB] IT's Your Life

멀티스레드
개념

멀티스레드
만드는 방법

멀티스레드
동기화

1 멀티 스레드 개념



1 멀티 스레드 개념

NAVER thread

통합검색 어학사전 ☐ 이미지 블로그 지식백과

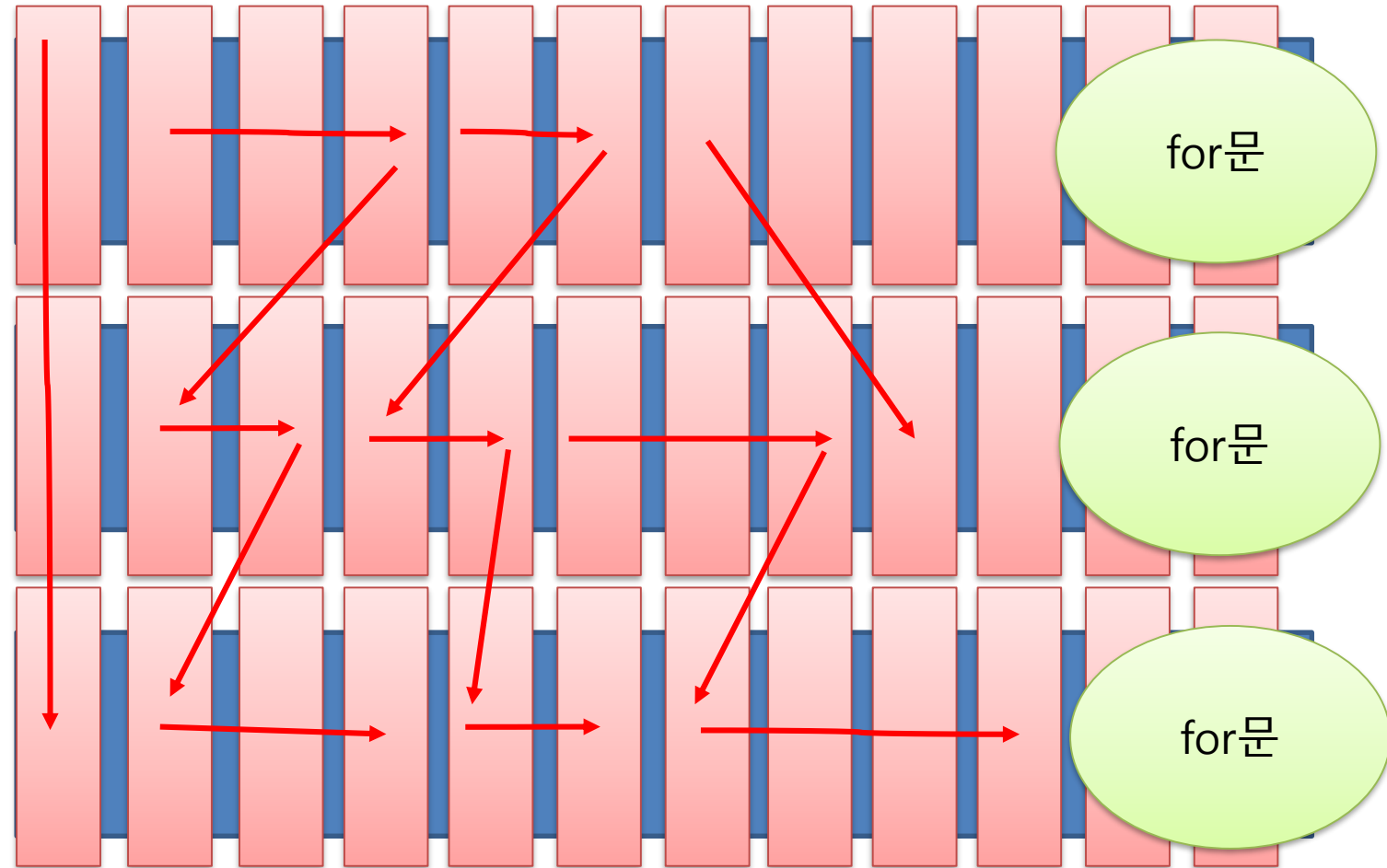
연관검색어 ☐ 스레드 코어 스레드 cpu 스레드 threaded
쓰레드 threading

어학사전

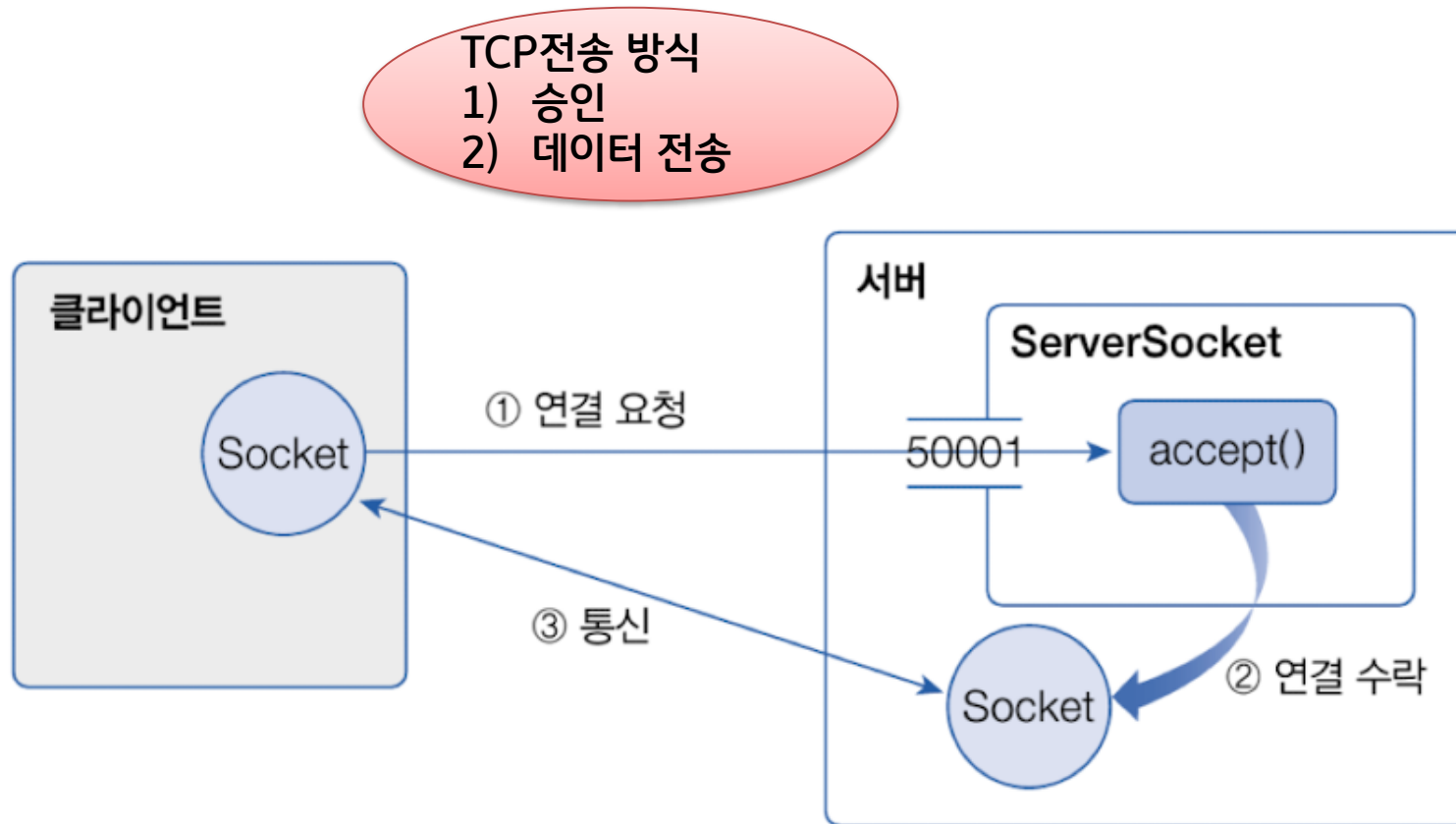
영어사전

thread 미국·영국 [θred] 영국식 ★★ 예문보기

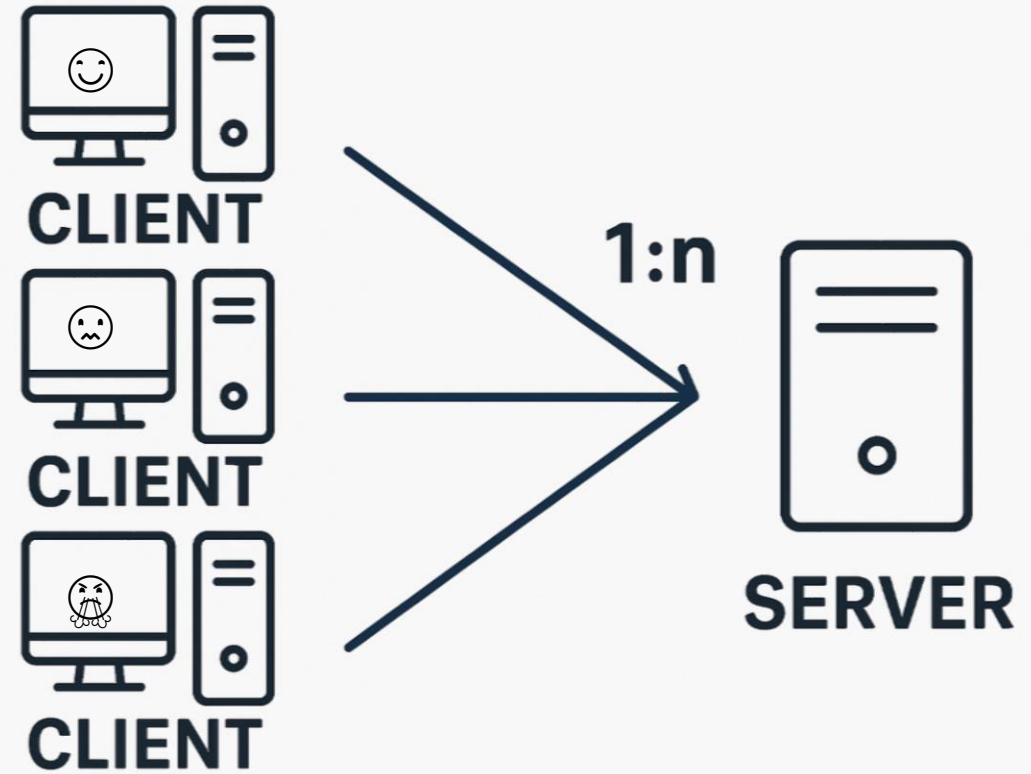
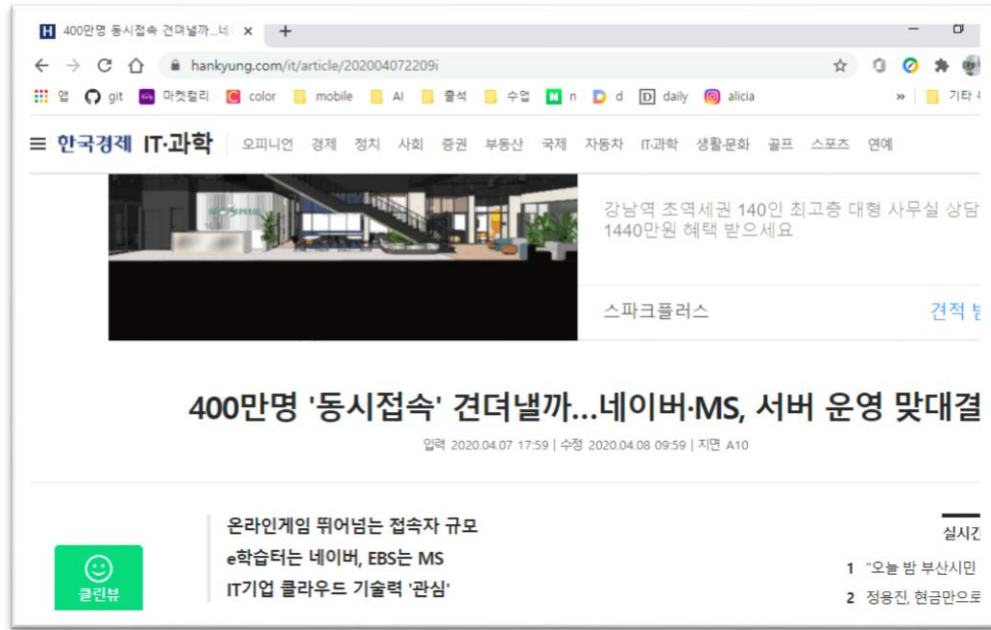
1. 실 2. (이야기 등의) 가닥 3. (실같이 가느다란) 줄기



1 멀티 스레드 개념



1 멀티 스레드 개념(1:다 관계)



1 멀티 스레드 개념

```
public class TCPServer {
    public static void main(String[] args) throws Exception {
        // Socket이 2개 필요
        // 클라이언트 연결 승인용: ServerSocket
        // 데이터 전송용: Socket
        // 예외처리: 외부의 자원을 연결하는 경우 (db, file, net, CPU)

        ServerSocket server = new ServerSocket(9100);
        System.out.println("TCP 서버 소켓 시작됨.");
        System.out.println("클라이언트의 연결을 기다리는 중...");

        int count = 0;
        while (true) {
            Socket socket = server.accept(); // 클라이언트 연결 수락
            count++;
            System.out.println("연결된 클라이언트 수: " + count);
            System.out.println("클라이언트와 연결 성공.");
        }
    }
}
```

```
public class TCPClients {
    public static void main(String[] args) throws Exception {
        for (int i = 0; i < 100; i++) {
            Socket socket = new Socket("localhost", 9100);
            System.out.println("클라이언트 " + i + ": 서버와 연결성공!!!!");
        }
    }
}
```

1 멀티 스레드 개념

✓ 멀티 프로세스와 멀티 스레드

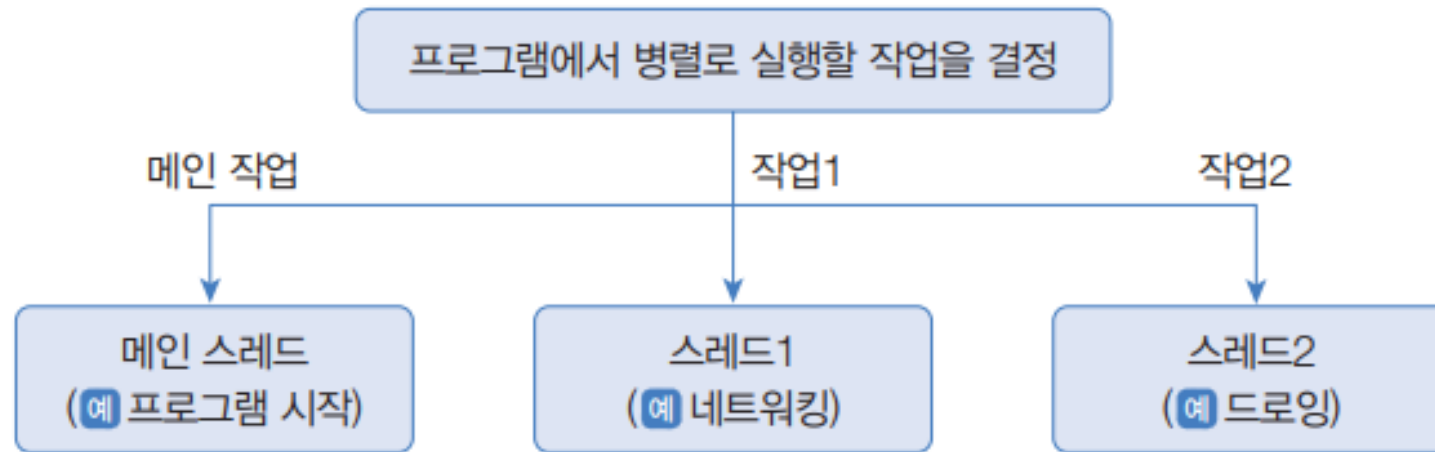
- 프로세스: 운영체제는 실행 중인 프로그램을 관리
- 멀티 스레드: 두 개의 코드 실행 흐름. 두 가지 이상의 작업을 처리



3 작업 스레드 생성과 실행

✓ 작업 스레드

- 멀티 스레드 프로그램을 개발 시 먼저 몇 개의 작업을 병렬로 실행할지 결정하고 각 작업별로 스레드를 생성



2 메인 스레드

✓ 메인 스레드

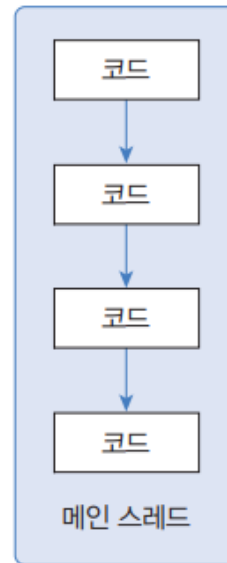
- 메인 스레드는 main() 메소드의 첫 코드부터 순차적으로 실행
- main() 메소드의 마지막 코드를 실행하거나 return 문을 만나면 실행을 종료
- 메인 스레드는 추가 작업 스레드들을 만들어서 실행시킬 수 있음
- 메인 스레드가 작업 스레드보다 먼저 종료되더라도 작업 스레드가 계속 실행 중이라면 프로세스는 종료되지 않음

```
public static void main(String[] args) {
    String data = null;
    if(...) {
    }
    while(...) {
    }
    System.out.println("...");
}
```

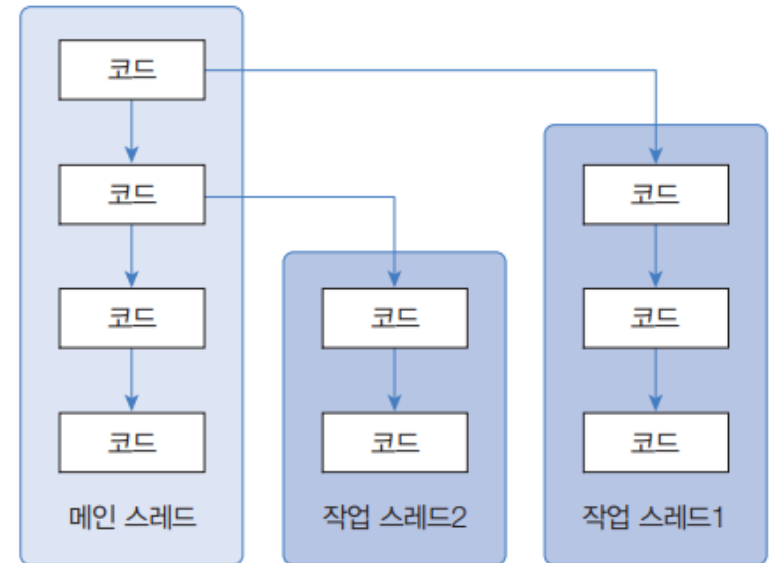
코드의 실행 흐름 → 메인 스레드



싱글 스레드 애플리케이션
프로세스



멀티 스레드 애플리케이션
프로세스



✓ 스레드 상태

○ 실행 대기 상태

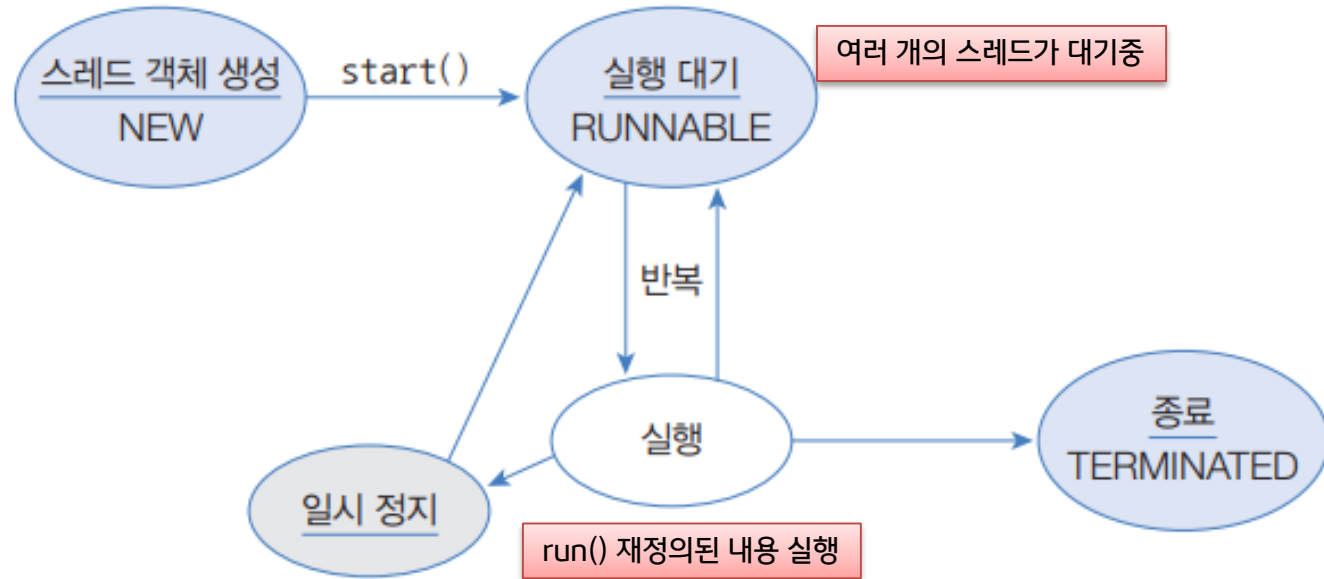
- 실행을 기다리고 있는 상태

○ 실행 상태

- CPU 스케줄링에 따라 CPU를 점유하고 run() 메소드를 실행.
- 스케줄링에 의해 다시 실행 대기 상태로 돌아갔다가 다른 스레드가 실행 상태 반복

○ 종료 상태:

- 실행 상태에서 run() 메소드가 종료
- 실행할 코드 없이 스레드의 실행을 멈춘 상태



스레드 프로그램의 순서

1. Thread 클래스 **상속을 받아서** 스레드클래스를 만든다.
 - 3개의 스레드를 동시에 실행하고 싶으면 Thread 클래스를 3개 만든다.

1. 멀티 스레드를 실행하고 싶은 main()을 포함하고 있는 클래스를 만든다.
 - a. new를 이용해서 Thread 객체 3개 생성
 - b. 생성한 Thread객체의 start()메서드 호출

Thread1
run() 오버라이드 :동시에 실행하고 싶은
내용을 넣어주세요.

Thread2
run() 오버라이드

Thread3
run() 오버라이드

```
t1 = new Thread1()
t2 = new Thread2()
t3 = new Thread3()
```

```
t1.start()
t2.start()
t3.start()
```

3 작업 스레드 생성과 실행

✓ Thread 클래스로 직접 생성

- 방법 1) Runnable 인터페이스에 따라 구현된 객체를 매개값으로 갖는 생성자를 호출

```
Thread thread = new Thread(Runnable target);
```

```
class Task implements Runnable {  
    @Override  
    public void run() {  
        //스레드가 실행할 코드  
    }  
}
```

Runnable
Interface에 따라 구
현한 Task클래스

3 작업 스레드 생성과 실행

✓ Thread 클래스로 직접 생성

- 방법 1) Runnable 구현 객체를 매개값으로 갖는 생성자를 호출

```
Runnable task= new Task();
Thread thread = new Thread(task);
```

Task를 Thread객체 생성
시 넣어줌.

```
Thread thread = new Thread( new Runnable() {
    @Override
    public void run() {
        //스레드가 실행할 코드
    }
});
```

익명클래스

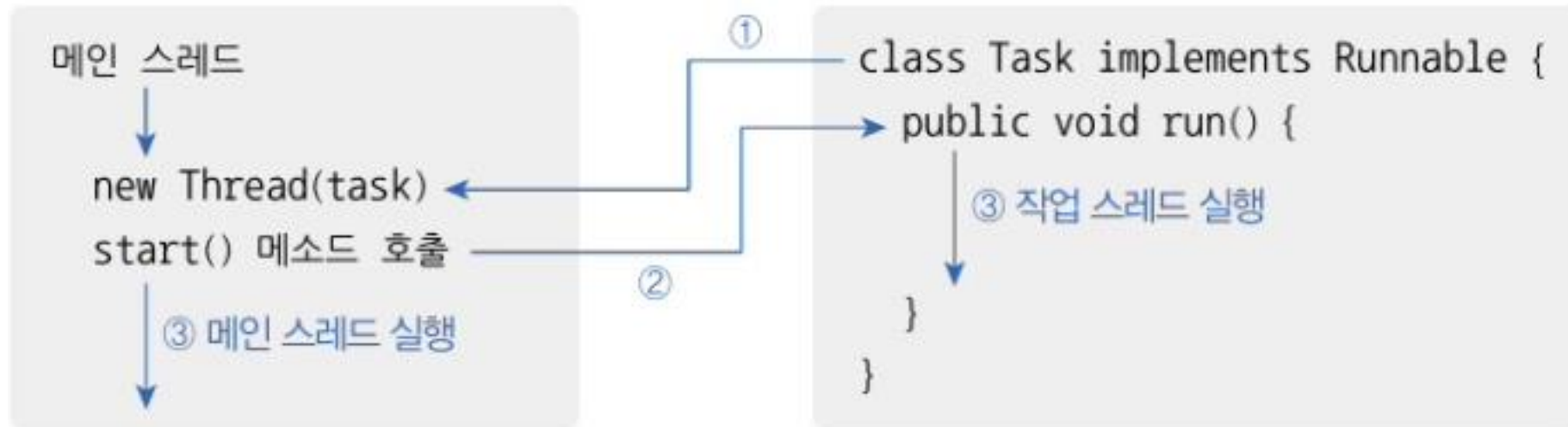
따로 Task를 구현하지 않고 이름
없는 클래스를 만들어 직접 입력값
으로 넣어주어도 됨.
→ 익명클래스(Task같은 이름이
없으므로)

```
thread.start();
```

3 작업 스레드 생성과 실행

✓ Thread 클래스로 직접 생성

- 방법 1) Runnable 구현 객체를 매개값으로 갖는 생성자를 호출



1. Thread객체를 만들어주고
2. start()메서드 호출하면
스레드 대기상태가 됨.

스레드가 실행되는 내용은 run()
메서드 안에 있는 처리 내용

3 **작업 스레드 생성과 실행** **BeepPrintExample.java**

```

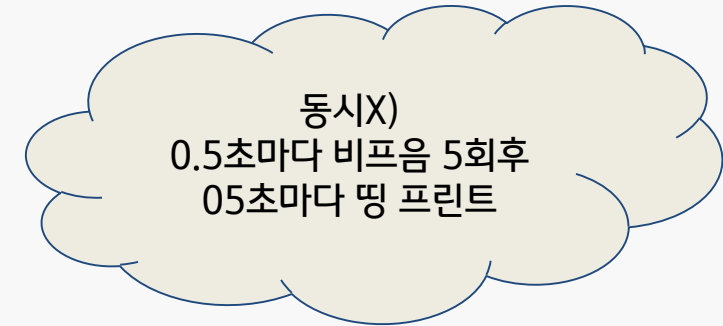
package ch14.sec03.exam01;

import java.awt.Toolkit;

public class BeepPrintExample {
    public static void main(String[] args) {
        Toolkit toolkit = Toolkit.getDefaultToolkit();
        for(int i=0; i<5; i++) {
            toolkit.beep();    // 비프음 발생
            try {
                Thread.sleep(500);
            } catch(Exception e) {
            }    // 0.5초간 일시 정지
        }

        for(int i=0; i<5; i++) {
            System.out.println("땡");
            try { Thread.sleep(500);
                } catch(Exception e) {}    // 0.5초간 일시 정지
        }
    }
}

```



땡
땡
땡
땡
땡

3 **작업 스레드 생성과 실행** **BeepPrintExample.java**

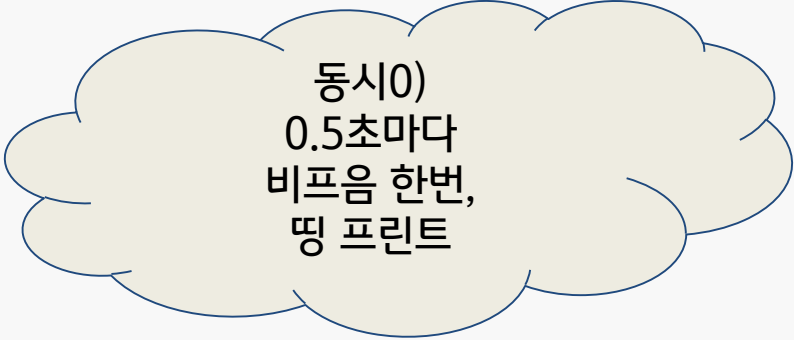
```
package ch14.sec03.exam02;
```

```
import java.awt.Toolkit;
```

```
public class BeepPrintExample {  
    public static void main(String[] args) {  
        Thread thread = new Thread(new Runnable() { // 작업 스레드 생성  
            @Override  
            public void run() {  
                Toolkit toolkit = Toolkit.getDefaultToolkit();  
                for(int i=0; i<5; i++) {  
                    toolkit.beep();  
                    try { Thread.sleep(500);  
                        } catch (Exception e) {  
                        }  
                }  
            }  
        });  
    }  
};
```

 익명클래스

```
thread.start(); // 작업 스레드 실행
```



동시0)
0.5초마다
비프음 한번,
띵 프린트

 BeepPrintExample.java

```
for(int i=0; i<5; i++) {  
    System.out.println("땡");  
    try { Thread.sleep(500); } catch(Exception e) {}  
}  
}
```

0F 0F 0F 0F 0F 0F

3 작업 스레드 생성과 실행

✓ Thread 자식 클래스로 생성

- 방법 2) Thread 클래스를 상속한 다음 run() 메소드를 재정의
 - 스레드가 실행할 코드를 작성
- 방법 3) 객체를 생성 혹은 Thread 익명 자식 객체를 사용 가능

```
public class WorkerThread extends Thread {  
    @Override  
    public void run() {  
        //스레드가 실행할 코드  
    }  
}
```

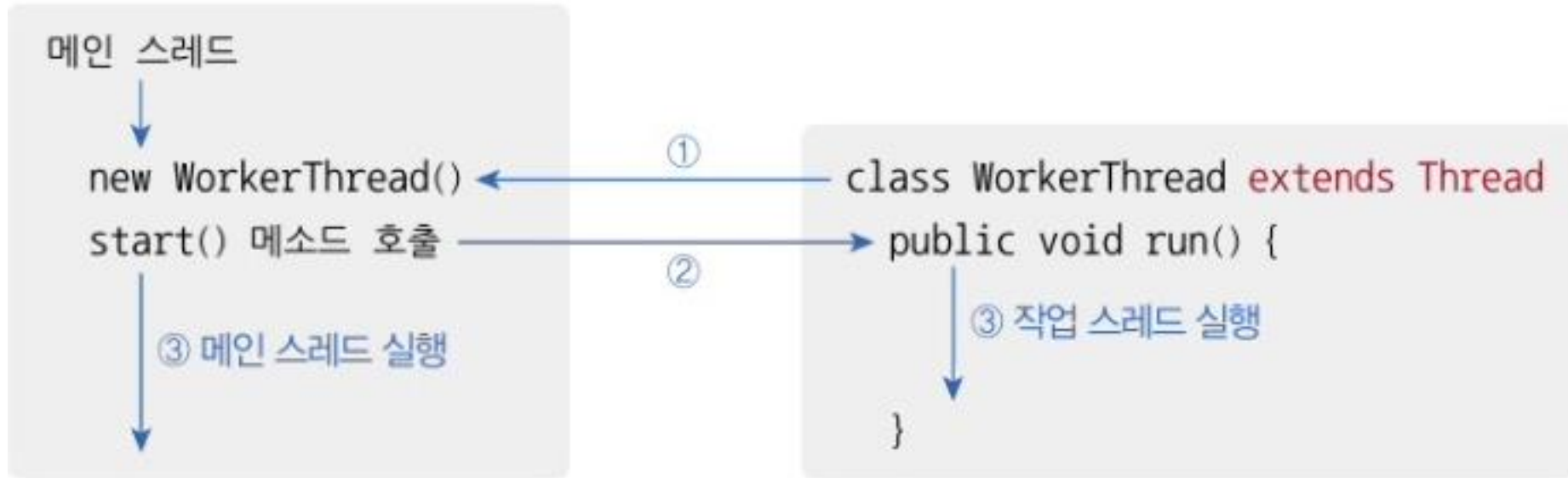
```
//스레드 객체 생성  
Thread thread = new WorkerThread();  
thread.start();
```

```
Thread thread = new Thread() {  
    @Override  
    public void run() {  
        //스레드가 실행할 코드  
    }  
};  
thread.start();
```

익명클래스

3 작업 스레드 생성과 실행

✓ Thread 자식 클래스로 생성



1. Thread객체를 만들어주고
2. start()메서드 호출하면
스레드 대기상태가 됨.

스레드가 실행되는 내용은 run()
메서드 안에 있는 처리 내용

3 **작업 스레드 생성과 실행** **BeepPrintExample.java**

```
package ch14.sec03.exam03;

import java.awt.Toolkit;

public class BeepPrintExample {
    public static void main(String[] args) {
        Thread thread = new Thread() {
            @Override
            public void run() {
                Toolkit toolkit = Toolkit.getDefaultToolkit();
                for(int i=0; i<5; i++) {
                    toolkit.beep();
                    try { Thread.sleep(500); } catch(Exception e) {}
                }
            }
        };
        thread.start();
    }
}
```

 익명클래스

 BeepPrintExample.java

```
for(int i=0; i<5; i++) {  
    System.out.println("땡");  
    try { Thread.sleep(500); } catch(Exception e) {}  
}  
}
```

0F 0F 0F 0F 0F 0F

✓ 작업 스레드의 이름

- 작업 스레드 이름을 Thread-n 대신 다른 이름으로 설정하려면 Thread 클래스의 setName() 메소드 사용

```
thread.setName("스레드 이름");
```

- 디버깅할 때 어떤 스레드가 작업을 하는지 조사하기 위해 주로 사용
- 어떤 스레드가 실행하고 있는지 확인하려면 정적 메소드인 currentThread()로 스레드 객체의 참조를 얻은 다음 getName() 메소드로 이름을 출력

```
Thread thread = Thread.currentThread();  
System.out.println(thread.getName());
```

ThreadNameExample.java

```
package ch14.sec04;

public class ThreadNameExample {
    public static void main(String[] args) {
        Thread mainThread = Thread.currentThread();

        System.out.println(mainThread.getName() + " 실행"); // 스레드 이름 리턴

        for(int i=0; i<3; i++) {
            Thread threadA = new Thread() {
                @Override
                public void run() {
                    System.out.println(getName() + " 실행"); // 스레드 이름 리턴
                }
            };
            threadA.start();
        }
    }
}
```

ThreadNameExample.java

```
Thread chatThread = new Thread() {  
    @Override  
    public void run() {  
        System.out.println(getName() + " 실행"); // 스레드 이름 리턴  
    }  
};  
chatThread.setName("chat-thread"); // 작업 스레드 이름 변경  
chatThread.start();  
}
```

```
main 실행  
Thread-1 실행  
Thread-0 실행  
chat-thread 실행  
Thread-2 실행
```


스레드 프로그램의 순서

Thread를 상속받아 run()재정의
의해 동시에 처리할 내용을 넣는다.

```
package thread;
```

```
public class Thread1 extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("증가: " + i);
        }
    }
}
```

```
package thread;
```

```
public class Thread2 extends Thread {
    @Override
    public void run() {
        for (int i = 100; i > 0; i--) {
            System.out.println("감소: " + i);
        }
    }
}
```

```
감소: 100
감소: 99
증가: 0
감소: 98
증가: 1
```

```
package thread;
```

```
public class ThreadUser {
    public static void main(String[] args) {
        Thread1 thread1 = new Thread1();
        Thread2 thread2 = new Thread2();
        thread1.start();
        thread2.start();
    }
}
```

Thread객체를 생성 후
각 스레드객체의 start()메서드를
호출하면, cpu스케줄러의 대기
기줄로 들어가고 순서가 되었을
때 run()메서드를 실행한다.

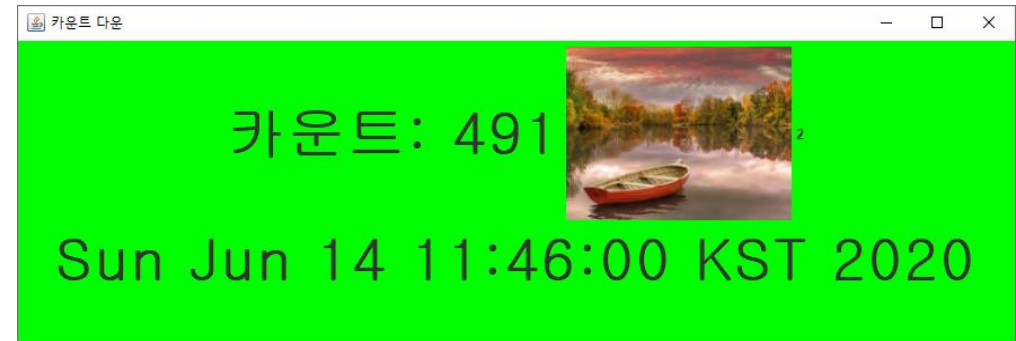
```
package thread;
```

```
public class Counter extends Thread {  
    @Override  
    public void run() {  
        for (int i = 500; i >= 0; i--) {  
            System.out.println("카운터: " + i);  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

```
package thread;
```

```
import java.util.Date;
```

```
public class Timer extends Thread {  
    @Override  
    public void run() {  
        for (int i = 0; i < 500; i++) {  
            Date date = new Date();  
            System.out.println("현재: " + date);  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



```
package thread;
```

```
public class Images extends Thread {  
    String[] s = {"감자.png", "고구마.png", "양파.png", "국수.png", "자장면.png"};  
  
    @Override  
    public void run() {  
        for (int i = 0; i < s.length; i++) {  
            System.out.println("음식이름: " + s[i]);  
            try {  
                Thread.sleep(3000);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

```
package thread;

public class ThreadUser2 {
    public static void main(String[] args) {
        Counter counter = new Counter();
        Timer timer = new Timer();
        Image image = new Image();

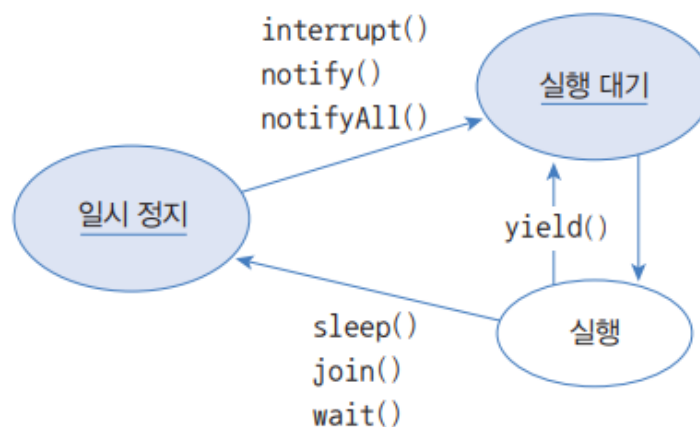
        counter.start();
        timer.start();
        image.start();
    }
}
```

음식이름: 감자.png
카운터: 500
현재: Wed Apr 23 19:01:34 KST 2025
카운터: 499
현재: Wed Apr 23 19:01:35 KST 2025
카운터: 498
현재: Wed Apr 23 19:01:36 KST 2025
음식이름: 고구마.png
카운터: 497
현재: Wed Apr 23 19:01:37 KST 2025
카운터: 496
현재: Wed Apr 23 19:01:38 KST 2025
카운터: 495
현재: Wed Apr 23 19:01:39 KST 2025

✓ 일시 정지 상태

- 스레드가 실행할 수 없는 상태
- 스레드가 다시 실행 상태로 가기 위해서는 일시 정지 상태에서 실행 대기 상태로 가야 함
- Thread 클래스의 `sleep()` 메소드:
 - 실행 중인 스레드를 일정 시간 멈추게 함
 - 매개값 단위는 밀리세컨드(1/1000)

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // interrupt() 메소드가 호출되면 실행
}
```



구분	메소드	설명
일시 정지로 보냄	<code>sleep(long millis)</code>	주어진 시간 동안 스레드를 일시 정지 상태로 만든다. 주어진 시간이 지나면 자동적으로 실행 대기 상태가 된다.
	<code>join()</code>	<code>join()</code> 메소드를 호출한 스레드는 일시 정지 상태가 된다. 실행 대기 상태가 되려면, <code>join()</code> 메소드를 가진 스레드가 종료되어야 한다.
	<code>wait()</code>	동기화 블록 내에서 스레드를 일시 정지 상태로 만든다.
일시 정지에서 벗어남	<code>interrupt()</code>	일시 정지 상태일 경우, <code>InterruptedException</code> 을 발생시켜 실행 대기 상태 또는 종료 상태로 만든다.
	<code>notify()</code> <code>notifyAll()</code>	<code>wait()</code> 메소드로 인해 일시 정지 상태인 스레드를 실행 대기 상태로 만든다.
실행 대기로 보냄	<code>yield()</code>	실행 상태에서 다른 스레드에게 실행을 양보하고 실행 대기 상태가 된다.

SleepExample.java

```
package ch14.sec05.exam01;

import java.awt.Toolkit;

public class SleepExample {
    public static void main(String[] args) {
        Toolkit toolkit = Toolkit.getDefaultToolkit();
        for(int i=0; i<10; i++) {
            toolkit.beep();
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
            }
        }
    }
}
```

- 자바에서는 초를 설정할 때 밀리 세컨즈 단위를 사용함. 1000밀리 세컨즈가 1초
- For문 실행할 때마다 3초씩 잠깐 실행하지 않음.
- Cpu를 연결하는 것이므로 예외처리 반드시 필요함.(try-catch)

✓ 다른 스레드의 종료를 기다림

- 어떤 스레드 A는 특정 스레드 B의 작업 결과를 바탕으로 동작
- 스레드 A는 스레드 B의 작업이 끝날 때까지 하는 일 없이 기다려야 함
→ 어떻게 스레드 B의 작업 종료를 인지할 수 있는가
- join() 메서드
 - 해당 스레드가 종료할 때까지 대기 하다가 스레드가 종료하면 실행을 재개



 SumThread.java

```
package thread2;

public class SumThread extends Thread {
    private long sum;

    public long getSum() {
        return sum;
    }

    public void setSum(long sum) {
        this.sum = sum;
    }

    @Override
    public void run() {
        for(int i=1; i<=100; i++) {
            sum+=i;
        }
    }
}
```

 SumThread.java

```
package thread2;

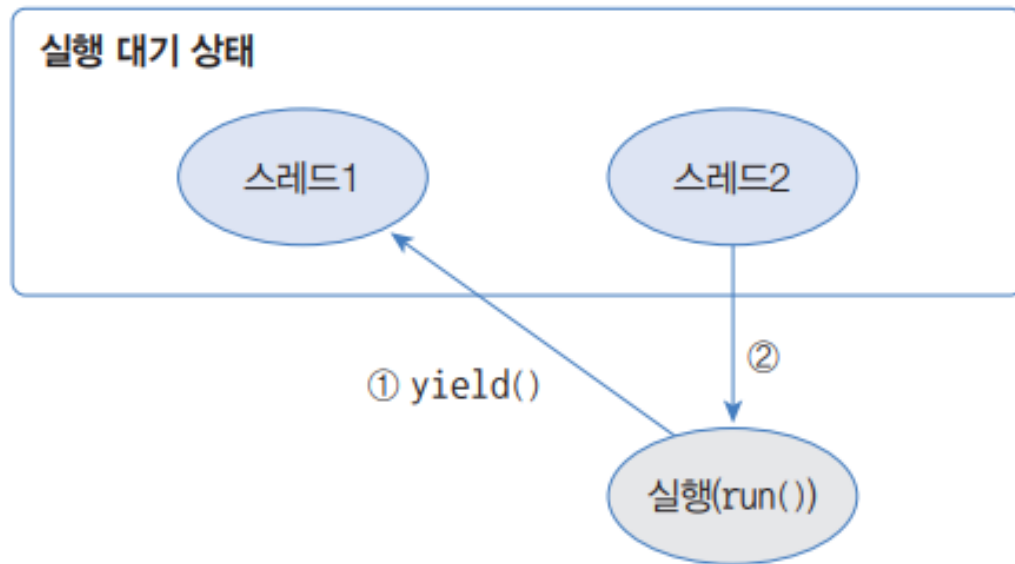
public class JoinExample {
    public static void main(String[] args) {
        SumThread sumThread = new SumThread();
        sumThread.start();
        try {
            sumThread.join();
        } catch (InterruptedException e) {
        }
        System.out.println("1~100 합: " + sumThread.getSum());
    }
}
```

1. main, sumThread 2개가 동시에 돌아가게 됨.
2. sumThread를 start()하면 run()과 main()이 동시에 실행됨.
3. 숫자를 다 더하는 sumThread가 먼저 끝나야 main스레드에서 총합을 프린트할 수 있음.
4. main스레드에서 총합을 프린트하기 전에 main스레드는 sumThread가 다 run()되기를 기다렸다가 프린트해야함.

1~100 합: 5050

✓ 다른 스레드에게 실행 양보

- `yield()` 메소드: 실행되는 스레드는 실행 대기 상태로 돌아가고, 다른 스레드가 실행되도록 양보



```
public void run() {  
    while(true) {  
        if(work) {  
            System.out.println("ThreadA 작업 내용");  
        } else {  
            Thread.yield();  
        }  
    }  
}
```

여러 스레드가 동시에 메서드를 호출하면 이상하게 값이 변하는 문제가 발생한다. 스레드는 동시에 처리되는 프로그램임.
잔고가 1000인 경우

1. 사람1이 withdraw(400)을 호출함. 아직 처리되지 않음.
2. 사람1의 인출 과정이 처리 중인데 그 사이 사람2가 withdraw(400)을 호출함.
3. 두 사람의 인출 과정이 끝나면 둘 다 시작이 잔고가 1000이었으므로 최종 잔고는 600이 됨.
4. 순서가 없었기 때문에 이상한 결과가 나온 것임.

```
class BankAccount {  
    private int balance = 1000;  
    public synchronized void  
        withdraw(int amount) {  
        if (balance >= amount )  
            System.out.println  
                Thread.currentThread.  
                    .getName() + '출금 시도  
                        중. 남은 잔액: '  
        } else {  
            System.out.println  
                Thread.currentThread.  
                    .getName() + '출금 실패.  
                        잔액 부족:.'  
        }  
    }  
}
```

withdraw(400)



사람1

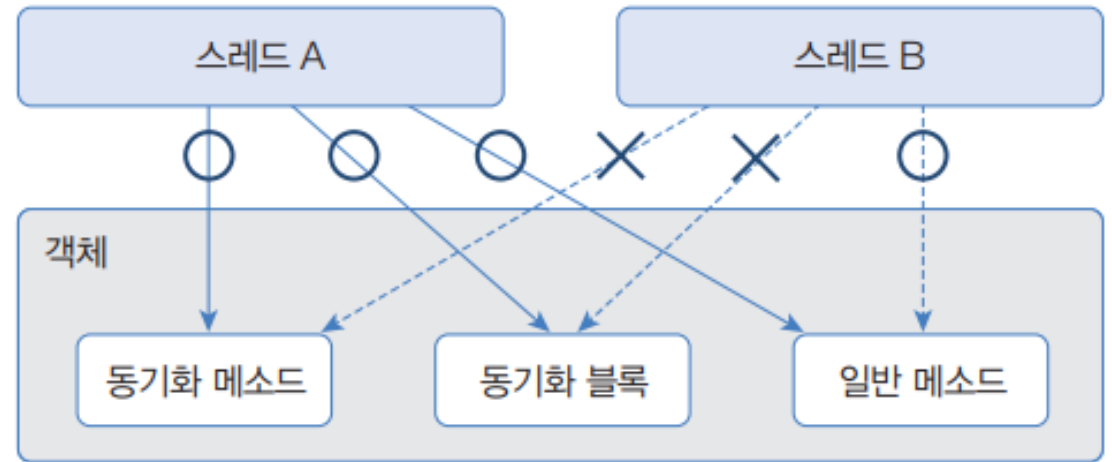
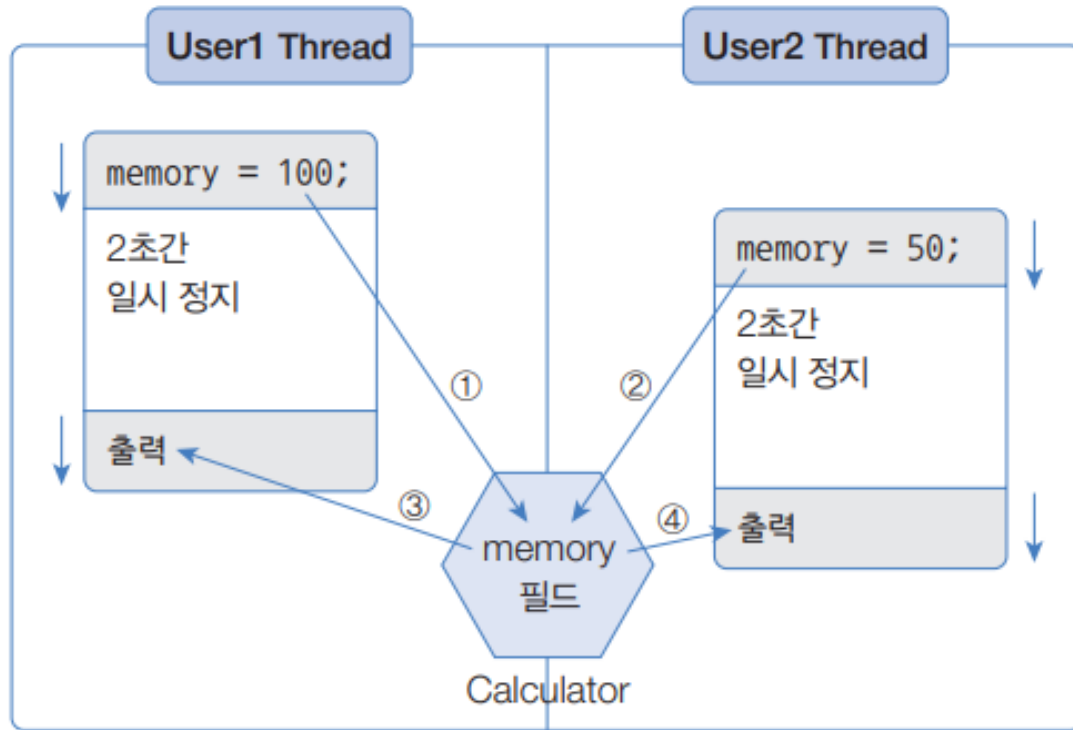
withdraw(400)



사람2

동기화 메소드와 블록

- 스레드 작업이 끝날 때까지 객체에 잠금을 걸어 스레드가 사용 중인 객체를 다른 스레드가 변경할 수 없게 함



✓ 동기화(순서를 정한다라는 의미 <-> 비동기) 메소드 및 블록 선언

- 인스턴스와 정적 메소드에 `synchronized` 키워드 붙임
- 동기화 메소드를 실행 즉시 객체는 잠금이 일어나고, 메소드 실행이 끝나면 잠금 풀림
- 메소드 일부 영역 실행 시 객체 잠금을 걸고 싶다면 동기화 블록을 만들

동기화 방법 1) 메서드

```
public synchronized void method() {
    //단 하나의 스레드만 실행하는 영역
}
```

동기화 방법 2) 객체

```
public void method () {
    //여러 스레드가 실행할 수 있는 영역

    synchronized(공유객체) {

        //단 하나의 스레드만 실행하는 영역
    }

    //여러 스레드가 실행할 수 있는 영역
}
```

synchronized

-> 스레드간 메서드, 변수 등을 접근시 **순서 제어** 가능

```
package thread2;

class BankAccount extends Thread {
    private int balance = 1000;

    public void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() + " 출금 시도 중...");
            balance -= amount;
            System.out.println(Thread.currentThread().getName() + " 출금 완료. 남은 잔액: " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() + " 출금 실패. 잔액 부족.");
        }
    }

    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            withdraw(400);
        }
    }
}
```

```
package thread2;

class BankAccount2 extends Thread {
    private int balance = 1000;

    public synchronized void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() + " 출금 시도 중...");
            balance -= amount;
            System.out.println(Thread.currentThread().getName() + " 출금 완료. 남은 잔액: " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() + " 출금 실패. 잔액 부족.");
        }
    }

    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            withdraw(400);
        }
    }
}
```

```
package thread2;

public class SyncTest {
    public static void main(String[] args) {

        BankAccount bankAccount = new BankAccount();

        Thread t1 = new Thread(bankAccount);
        Thread t2 = new Thread(bankAccount);

        t1.start();
        t2.start();
    }
}
/*
Thread-1 출금 시도 중...
Thread-2 출금 시도 중...
Thread-2 출금 완료. 남은 잔액: 200
Thread-1 출금 완료. 남은 잔액: 600
Thread-2 출금 실패. 잔액 부족.
Thread-2 출금 실패. 잔액 부족.
Thread-1 출금 실패. 잔액 부족.
Thread-1 출금 실패. 잔액 부족.
*/
```

Synchronized X

-> 제대로 동작하지 않음

```
package thread2;

public class SyncTest2 {
    public static void main(String[] args) {
        BankAccount2 bankAccount = new BankAccount2();

        Thread t1 = new Thread(bankAccount);
        Thread t2 = new Thread(bankAccount);

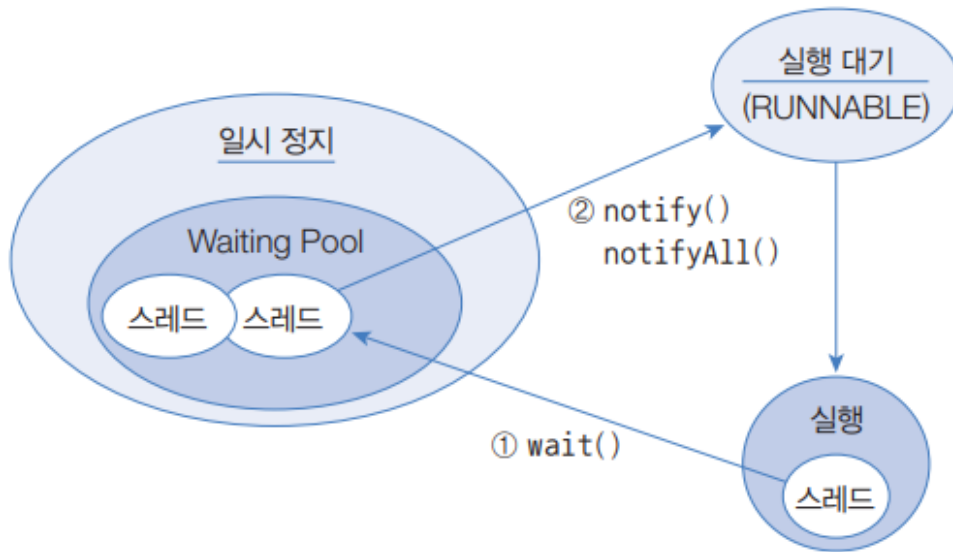
        t1.start();
        t2.start();
    }
}
/*
Thread-1 출금 시도 중...
Thread-1 출금 완료. 남은 잔액: 600
Thread-1 출금 시도 중...
Thread-1 출금 완료. 남은 잔액: 200
Thread-1 출금 실패. 잔액 부족.
Thread-2 출금 실패. 잔액 부족.
Thread-2 출금 실패. 잔액 부족.
Thread-2 출금 실패. 잔액 부족.
Thread-2 출금 실패. 잔액 부족.
*/
```

Synchronized O

-> 제대로 동작함

✔ wait()과 notify()를 이용한 스레드 제어

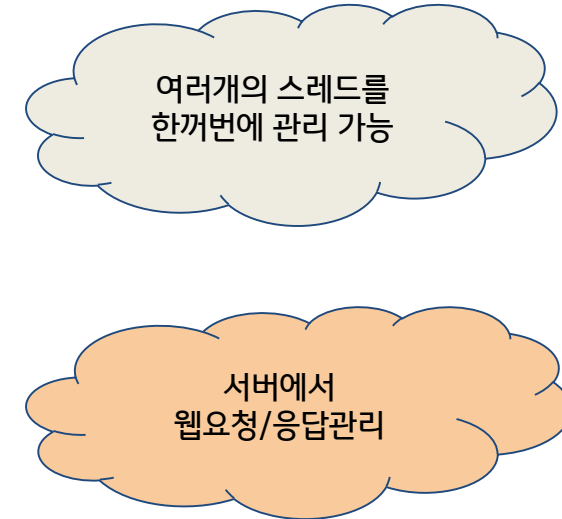
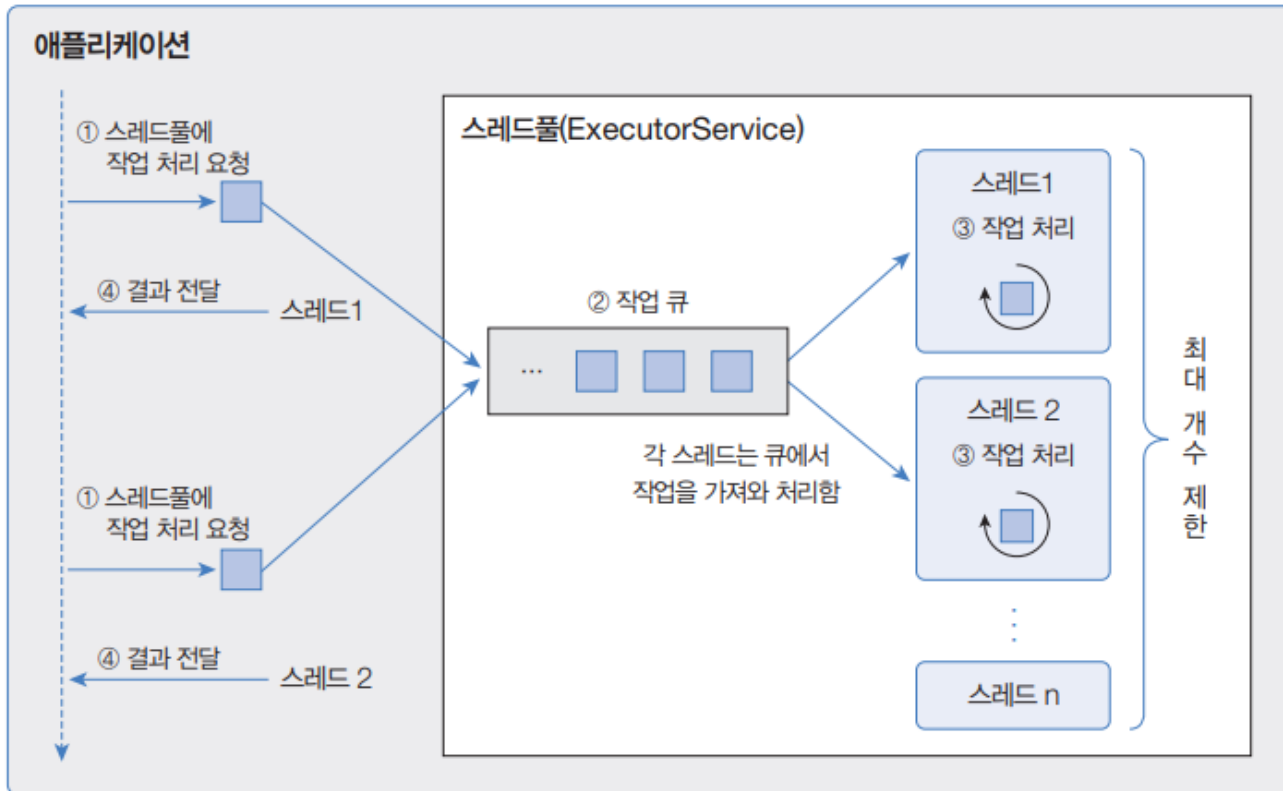
- 두 스레드 교대 실행 시 공유 객체는 두 스레드가 작업할 내용을 각각 동기화 메소드로 정함
- 한 스레드 작업 완료 시 notify() 메소드를 호출해 일시 정지 상태에 있는 다른 스레드를 실행 대기 상태로 만들고, wait() 메소드를 호출하여 자신은 일시 정지 상태로 만들



- `notify()`: `wait()`에 의해 일시 정지된 스레드 중 한 개를 실행 대기 상태로 만들
- `notifyAll()`: `wait()`에 의해 일시 정지된 모든 스레드를 실행 대기 상태로 만들

✓ 스레드풀로 작업 처리 제한하기

- 미리 사용할 스레드를 만들어두고 사용
- 작업 처리에 사용되는 스레드 개수를 제한하고 작업 큐에 들어오는 작업들을 스레드가 하나씩 맡아 처리하는 방식
- 작업 처리가 끝난 스레드는 다시 작업 큐에서 새로운 작업을 가져와 처리
- 작업량이 증가해도 스레드의 개수가 늘어나지 않아 애플리케이션의 성능의 급격한 저하 방지



✓ 스레드풀 생성

- java.util.concurrent 패키지에서 ExecutorService 인터페이스와 Executors 클래스를 제공
- Executors의 다음 두 정적 메소드를 이용하면 스레드풀인 ExecutorService 구현 객체를 만들 수 있음

메소드명(매개변수)	초기 수	코어 수	최대 수
newCachedThreadPool()	0	0	Integer.MAX_VALUE
newFixedThreadPool(int nThreads)	0	생성된 수	nThreads

- 초기 수: 스레드풀이 생성될 때 기본적으로 생성되는 스레드 수
- 코어 수: 스레드가 증가된 후 사용되지 않는 스레드를 제거할 때 최소한 풀에서 유지하는 스레드 수
- 최대 수: 증가되는 스레드의 한도 수

```
ExecutorService executorService = Executors.newCachedThreadPool();
```

```
ExecutorService executorService = Executors.newFixedThreadPool(5);
```